

DeepSeek-V4.1-Flash

架构精读与

KV Cache 技术脉络

从 Transformer 的历史记忆
到 CED、CSA2、FP4 与 Bounded Replay

用相对便宜的路径构造历史信息，
用跨层共享的方式保存历史信息，
让当前生成位置完成完整的深层计算。

报告中的三个核心量化口径

8B / 16B

Prefill / Decode
激活参数规模

890 bytes/token

全部独立 global KV
的每 token 存储量

1/4 与 1/8

Global KV / Persistent KV
相对 V4-Flash 的比例

中文技术阅读笔记·完整解答增补版

基于上传的技术报告及相关前作整理，非官方文档。

2026 年 9 月 11 日

阅读范围与来源

这份笔记围绕三个问题展开：DeepSeek-V4.1-Flash 为什么能够改善长上下文推理效率，为什么 prefill 与 decode 的激活参数规模不同，以及 global KV 的四倍压缩和 persistent KV 的八倍压缩分别是怎样实现的。

正文之前新增了 KV cache 基础回顾，从标准 Transformer 的 Q/K/V、自回归增量执行和逐层状态开始，解释为什么缓存会随着上下文、层数、KV heads 和并发规模迅速膨胀。随后保留上一轮解答的完整技术讨论、计算示例和数学表达，并将公式、章节与引用统一编号。

主要依据是用户上传的 51 页报告《DeepSeek-V4.1-Flash: Pushing the Limits of KV Cache Compression》，文献编号为 [1]。文中标注“原报告”时，页码均指该文件的页码，而不是这份笔记的页码。核心配置、部署策略、精度选择和实验结论以该报告为准。

历史背景与前作来自原始论文及官方文档，列于文末参考文献。MHA、MQA、GQA、MLA、YOCO 等基础介绍属于补充背景；V4 的 compressor、mHC、Engram 和 DSpark 的前作机制用于解释本报告继承的设计，不应被视为本报告逐项重新证明的结论。

推导与示意包括缓存容量公式、假设模型的 GiB 计算、层—token 工作量估算、候选索引数量和 890 bytes/token 的复原。它们会明确标为记账推导或简化表达，不是实测延迟或吞吐数据。

两个阅读提醒。“每层都保存历史”不意味着每层保存的 KV 数值完全相同；“上下文很长但 decode FLOPs 增长很小”也不等于真实推理延迟严格恒定。前者是架构依赖问题，后者还涉及硬件、数据搬运与系统负载。

排版校注。上一版对 Engram 维度的表述略简：原报告实际是**每个 N-gram order 的总 embedding dimension 为 2048**，而不是三个 orders 合计 2048。本稿在相应位置明确这一口径，其余技术讨论保持原有范围。[1, §2.4.2, p. 13]

目录

阅读范围与来源	i
第 1 章 前置回顾：KV cache 为什么会变得这么大？	1
1.1 先把 Q、K、V 和“历史记忆”对应起来	1
1.2 KV cache 原本是在节省计算，而不是制造额外负担	1
1.3 为什么同一个历史 token 要在每一层分别保存？	2
1.4 一笔完整的容量账：线性增长为什么仍然可怕？	3
1.5 分清三个增长：缓存容量、计算量、数据读取量	4
1.6 历史上怎样优化 KV？各自又没有解决什么？	5
1.7 带着这张“乘法账单”进入 V4.1	7
第 2 章 整体图景：它重新划分了什么？	8
2.1 它到底是一个什么模型？	8
2.2 先区分三种“记忆”	9
2.3 从 DeepSeek 之前的工作走到这里	9
第 3 章 CED：为什么历史 token 可以少算一半层？	11
3.1 普通 decoder-only 模型为什么不能随便跳过上半层？	11
3.2 关键改动：decoder 的 global KV 不再来自 decoder 自己	11
3.3 为什么 prefill 可以跳过，decode 却不能跳过？	11
3.4 它与传统 encoder-decoder、YOCO 有什么区别？	12
第 4 章 CSA2：究竟共享了什么，又保留了什么？	13
4.1 一次 CSA2 attention 的计算过程	13
4.2 Compressor：压缩的是内部表示，不是文本摘要	13
4.3 Indexer：先便宜地筛选，再进行主 attention	14
4.4 Full、Reindex、Reuse：三种模式的区别	14
4.5 具体到 40 层，究竟有几套 global KV？	15
4.6 Hierarchical Sparse Indexer：限制后续索引的搜索范围	16
4.7 低秩 Query 与分组 Output Projection	17
第 5 章 三个辅助架构：mHC、Engram 与 DSpark	18
5.1 Single-Pass mHC：多条残差通路，不多搬运数据	18

5.2	Engram: 用查表扩展容量, 而非让所有知识都经过专家计算	19
5.3	DSpark: 一次提出多个候选, 但不盲目验证所有候选	20
第 6 章	FP4 KV: 低精度存储, 不等于全部 attention 都用 FP4 算	22
6.1	两种 FP4, 用途不同	22
6.2	Main KV 的具体格式与数值范围	22
6.3	为什么 SWA 不也压成 FP4?	23
第 7 章	SWA Bounded Replay: 用近似重计算替代长期局部缓存	24
7.1	128-token 窗口, 为什么恢复时可能需要重算更多?	24
7.2	Bounded Replay 做了什么近似?	24
7.3	Encoder Bounded Replay: 全局缓存命中, 局部缓存缺失	24
7.4	Decoder Bounded Replay: 让 prefill 提前结束成为可用路径	25
第 8 章	其余组件: MoE、多模态、优化器与系统实现	26
8.1	MoE: 大容量低激活的基础, 但不是 8B/16B 差异的来源	26
8.2	多模态输入: 视觉 token 数量也在压缩	26
8.3	优化器: 主要是在降低训练这些结构的成本	27
8.4	系统实现: 把结构节省兑现成实际速度	27
第 9 章	三个核心问题: 加速、激活参数与 KV 缩减	28
9.1	问题一: 它怎么大幅提高 inference 速度?	28
9.2	问题二: 为什么 decode 和 prefill 的激活参数可以不一样?	29
9.3	问题三: 为什么 KV 能缩这么多, 究竟是 1/4 还是 1/8?	29
9.4	最后的判断: 真正值得关注的创新是什么?	32
附录 A	符号、公式与原报告定位	33
A.1	主要符号与统计口径	33
A.2	关键公式快速定位	34
A.3	原报告的阅读路径	34
参考文献		35

第 1 章 前置回顾：KV cache 为什么会变得这么大？

KV cache 的容量通常随上下文长度**线性增长**，不是二次增长。之所以看起来增长得特别快，是因为每个位置不是只存一个 token ID，而是要在许多层中分别保存高维的 K/V，再乘上并发请求数和数值精度。

1.1 先把 Q、K、V 和“历史记忆”对应起来

在标准自注意力中，某一层会把输入 hidden states 投影为 query、key 和 value。为方便说明，本章用 $h_t^{(\ell)}$ 表示第 ℓ 层 attention 投影处、位置 t 的行向量输入，省略归一化和位置变换：

$$q_t^{(\ell)} = h_t^{(\ell)} W_Q^{(\ell)}, \quad k_t^{(\ell)} = h_t^{(\ell)} W_K^{(\ell)}, \quad v_t^{(\ell)} = h_t^{(\ell)} W_V^{(\ell)}. \quad (1.1)$$

对单个 head，位置 t 的因果注意力可以写成：

$$\alpha_{t,s}^{(\ell)} = \frac{\exp(q_t^{(\ell)} (k_s^{(\ell)})^\top / \sqrt{d_h})}{\sum_{u=1}^t \exp(q_t^{(\ell)} (k_u^{(\ell)})^\top / \sqrt{d_h})}, \quad s \leq t, \quad (1.2)$$

$$o_t^{(\ell)} = \sum_{s=1}^t \alpha_{t,s}^{(\ell)} v_s^{(\ell)}. \quad (1.3)$$

批量写法则是熟悉的：

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_h}} + M_{\text{causal}}\right) V. \quad (1.4)$$

这里的 M_{causal} 屏蔽未来位置。标准 Transformer 的 scaled dot-product attention 和多头结构见原始论文。[2, §3.2]

一个有用但并不严格等同于语义的比喻是：**Q** 表示当前想查询什么，**K** 表示历史内容如何被匹配，**V** 表示匹配后读取什么。新位置的 query 要和所有允许访问的历史 keys 比较，再混合对应 values。正是这种逐位置、内容可寻址的历史访问，令模型能够直接检索很久之前的信息。

多头注意力把这个操作放到不同投影子空间中并行执行。**Query heads 的数量与 KV heads 的数量不一定相同**：标准 MHA 中相同，MQA/GQA 则让多个 query heads 共享 KV heads。[3, 4]

1.2 KV cache 原本是在节省计算，而不是制造额外负担

1.2.1 不用缓存：每生成一步都从头计算前缀

假设 prompt 有 N 个 token，随后不断生成新 token。如果每次预测都把当前完整前缀重新送进网络，那么此前已经计算过的 token 又会经过各层，重新生成其 hidden states 和 K/V。这个朴素实现虽然可以得到预测，但大量工作是重复的。

对于**固定参数、固定位置机制、前缀内容不变的因果模型**，位置 s 的状态只依赖 $x_{1:s}$ ，不会因为后来追加 $x_{s+1}, x_{s+2}, \dots$ 而改变。因此，以前算好的各层 K/V 可以继续复用。标准自回归缓存正是利用这个因果不变性。[3, 5]

1.2.2 使用缓存：只新增当前位置的状态

对每层维护两个不断追加的张量：

$$K_{1:t}^{(\ell)} = \begin{bmatrix} K_{1:t-1}^{(\ell)} \\ k_t^{(\ell)} \end{bmatrix}, \quad V_{1:t}^{(\ell)} = \begin{bmatrix} V_{1:t-1}^{(\ell)} \\ v_t^{(\ell)} \end{bmatrix}. \quad (1.5)$$

Prefill 并行处理已知的 prompt，生成其缓存。之后的常规 decode 每次处理一个新位置：在第 1 层生成当前 K/V、读取第 1 层的历史缓存，再进入第 2 层，以此类推。**旧位置在同一层的 K/V 通常不需要重算；新增位置仍然要经过整个网络。** [5]

这里按“处理 x_t ，用输出预测 x_{t+1} ”计数。第一个输出 token 可以由 prompt 最后位置的输出分布采样；下一步再把这个已采样 token 作为新的输入位置处理，不必把“预测的 token”和“当前被送入网络的 token”混为一谈。

1.2.3 为什么通常缓存 KV，而不缓存历史 Q?

从式 (1.3) 可以直接看出，计算当前输出需要的是当前 q_t 和历史 (k_s, v_s) 。过去的 q_s 已经完成了过去位置的查询，不会被未来 query 直接使用；过去的 attention output 也不能代替当前输出，因为当前 query 导致的权重分布不同。

因此，普通增量自注意力长期保留的是 K/V，**不是所有历史 Q，也不是整个历史 attention score 矩阵**。一些特殊推理或训练算法可能保留额外状态，但那不是标准 KV cache 的基本需求。[5]

1.3 为什么同一个历史 token 要在每一层分别保存？

1.3.1 “同一段历史”不等于“相同的向量”

取历史中的同一个位置 s 。它在第 3 层和第 30 层的 attention 输入一般不同，各层的投影参数也不同，因此：

$$(k_s^{(3)}, v_s^{(3)}) \neq (k_s^{(30)}, v_s^{(30)}) \quad \text{一般成立。} \quad (1.6)$$

也就是说，各层保存的是**同一历史位置在不同计算深度、不同特征空间中的表示**。第 30 层当前 token 的 query 是为该层的历史表示空间训练的，不能在不改变模型函数的情况下，直接换成第 3 层的 KV。这个结论来自分层投影与非线性状态更新的结构，而不是缓存实现中的偶然缺陷。[2, §3][6, §2.1]

即便把不同层的 W_K, W_V 强行设成相同，输入 hidden states 仍可能不同；**共享投影权重也不自动等于共享 KV 激活**。

1.3.2 为什么不能只存一份 token embedding，然后用的时候再变换？

因为高层 K/V 不是 token ID 的固定函数。它依赖之前层的非线性计算、attention 聚合以及当前位置之前的上下文。即使两处 token ID 相同，只要前缀不同，深层状态通常就不同。

若只存原始 embedding，需要时再恢复第 30 层 KV，往往意味着重新运行之前的许多层及其依赖。这会把原本用缓存省掉的计算重新带回来。若只存每层 hidden states，则虽可能省掉某些投影后的表示，但仍然要保存逐层状态，并支付读取时重新投影的成本；具体是否划算还取决于表示维度与架构。

传统 KV cache 可以沿时间复用同一层的旧状态，却不能默认沿深度复用另一层的状态。跨层共享是需要改变并训练计算图的架构设计，不是给已有缓存做一次普通去重。

跨层 attention 与 YOCO 正是明确改写这种依赖关系的研究方向。后面的 CED 和 CSA2 进一步把 global memory 共享与 layer-local memory 分开。[9, 10][1, §2.2–2.3, pp. 9–11]

1.4 一笔完整的容量账：线性增长为什么仍然可怕？

设模型有 L 层，每层 h_{KV} 个 KV heads，K/V 每个 head 均为 d_h 维，每个数占 s bytes。对 B 个等长、长度为 T 的请求，普通 KV 张量的逻辑形状可以写成：

$$K^{(\ell)}, V^{(\ell)} \in \mathbb{R}^{B \times h_{KV} \times T \times d_h}. \quad (1.7)$$

不计 scale、对齐、分片复制及管理元数据，两个缓存的总容量是：

$$M_{KV} = 2sBLTh_{KV}d_h. \quad (1.8)$$

对不同请求长度 T_b ，则为：

$$M_{KV} = 2sLh_{KV}d_h \sum_{b=1}^B T_b. \quad (1.9)$$

这是由缓存张量形状得到的**记账推导**。前面的 2 表示两套独立的 K 与 V；采用联合 latent 或 shared K/V 表示时，不能不加区分地继续乘这个 2。标准 MHA 的逐 token 缓存元素计数也见 DeepSeek-V2 的背景介绍。[6, §2.1.1]

1.4.1 假设模型示例：每个 token 可以对应半 MiB 缓存

考虑一个用于记账的假设模型，不对应本报告的具体模型：

$$L = 32, \quad d_{\text{model}} = 4096, \quad h_Q = h_{KV} = 32, \quad d_h = 128, \quad s = 2. \quad (1.10)$$

则一个请求每多处理一个 token，所有层合计新增：

$$\Delta M = 2 \times 2 \times 32 \times 32 \times 128 = 524,288 \text{ bytes} = 512 \text{ KiB}. \quad (1.11)$$

同样的层数和 head dimension，如果改为 8 个 KV heads 的 GQA，新增量为 128 KiB；如果是 1 个 KV head 的 MQA，则为 16 KiB。这里只比较理论存储，不声称三种模型的能力相同。

表 1.1 同一假设配置下，单请求 KV 容量的记账比较

已处理的上下文长度	MHA, 32 KV heads	GQA, 8 KV heads	MQA, 1 KV head
4K = 4,096	2 GiB	0.5 GiB	0.0625 GiB
32K = 32,768	16 GiB	4 GiB	0.5 GiB
128K = 131,072	64 GiB	16 GiB	2 GiB
1,000,000	488.28 GiB	122.07 GiB	15.26 GiB

这里 1 GiB = 2^{30} bytes。例如 16 个并发请求，每个上下文为 32K，上述 MHA 缓存合计为 256 GiB；GQA 也有 64 GiB。尚未计入模型权重、计算 workspace 和其他状态。

1.4.2 模型权重与 KV 的增长方式不同

同一个模型的权重通常可以在请求之间共享，模型不变时不会因为对话多一个 token 而增长。KV 则是**输入相关、位置相关、请求相关的激活状态**：上下文更长、并发更多，容量就继续增加。

精确共享相同前缀能够减少多请求中的重复物理块，但不能默认任意请求都共享缓存。相同的词或句子出现在不同前缀之后，也不等于得到相同的深层 KV。PagedAttention 的块共享和 copy-on-write 是这种**同层、同前缀、跨请求复用**，不是**同一请求、不同层之间的架构共享**。[17]

1.5 分清三个增长：缓存容量、计算量、数据读取量

1.5.1 KV cache 是线性的；dense attention 的全序列计算是二次的

对长度为 T 的因果序列，每层每个 head 有约：

$$\sum_{t=1}^T t = \frac{T(T+1)}{2} \quad (1.12)$$

个 query-key 配对。因此，忽略投影和 FFN，dense prefill 的 attention 计算量按下式增长：

$$F_{\text{prefill,attn}} = \Theta(BLh_Q T^2 d_h). \quad (1.13)$$

但缓存只保留各位置的 K/V，不需要长期保存每一对位置的分数，所以：

$$M_{\text{KV}} = \Theta(BLh_{\text{KV}} T d_h s). \quad (1.14)$$

单次普通 decode 只有当前 query，需要与历史位置匹配，attention 计算则大致为：

$$F_{\text{decode,attn}} = \Theta(BLh_Q T d_h). \quad (1.15)$$

以上是根据注意力表达式得到的渐近分析。**不要把 prefill 的 $T \times T$ attention 矩阵与跨步骤保留的 KV cache 混为一谈**。[2, 16]

1.5.2 每步只新增很少，却可能反复读取大量历史

虽然每次 decode 只追加一个位置的 KV，但 dense attention 通常仍需扫描已有的 K/V。理想化的单次扫描中，跨全部层的 **KV 逻辑阅读量** 约为：

$$D_{KV, \text{step}} \approx 2sLh_{KV}d_h \sum_{b=1}^B T_b. \quad (1.16)$$

真实 HBM 流量还受片上缓存命中、head 复用、kernel tiling、分片与额外读取影响，不能把这条逻辑记账式当成所有硬件上的严格 HBM 下界。MQA 的原始动机就包括减少增量解码时反复加载 K/V 的带宽成本。[3]

用一个简化 **roofline** 视角描述算子延迟，有：

$$\tau \gtrsim \max\left(\frac{F}{P_{\text{eff}}}, \frac{D}{\beta_{\text{eff}}}\right), \quad (1.17)$$

其中 F 是运算量， D 是数据搬运量， P_{eff} 与 β_{eff} 分别是有效算力和带宽。这是性能分析的示意，不是本报告给出的实测模型。它说明：更小的缓存能减轻容量与带宽压力，但不保证端到端延迟按相同比例下降；权重读取、反量化、通信和其他算子也可能成为瓶颈。

1.6 历史上怎样优化 KV？各自又没有解决什么？

1.6.1 MHA → MQA → GQA：减少每层保存多少个 KV heads

原始 MHA 为多个 heads 提供各自的 K/V 投影。MQA 保留多个 query heads，但让它们共享一套 K/V；GQA 位于两者之间，让每一组 query heads 共享一套 K/V。[2-4]

$$h_{KV} = \begin{cases} h_Q, & \text{MHA,} \\ G, & 1 < G < h_Q, \text{ GQA,} \\ 1, & \text{MQA.} \end{cases} \quad (1.18)$$

从式 (1.8) 看，这直接缩小 h_{KV} 。但层数 L 和历史长度 T 仍然存在。MQA/GQA 的“共享”首先是同一层内部、不同 query heads 之间的共享，不是跨层共享。GQA 的设计目标是在质量与解码效率之间取得更好的折中。[4]

1.6.2 MLA：把每个位置的多头 KV 联合压到低维 latent

MLA 不只是把 KV heads 分组，而是缓存可供多头使用的共同低维表示，再通过代数吸收减少展开 K/V 的需要。考虑解耦的 RoPE 分量，其典型记账可写成：

$$M_{\text{MLA}} \approx sBLT(d_c + d_R), \quad (1.19)$$

其中 d_c 是 KV latent 维度， d_R 是额外位置 key 的维度。这条式子假设二者采用相同精度，省略额外布局开销。MLA 压缩的是每位置的表示宽度，通常仍然每层、每位置保留一份 latent。具体数学机制见第 2 章。[6, §2.1]

1.6.3 SWA、稀疏选择与时间压缩：三者不能混为一谈

SWA 只直接访问最近 W 个位置，可用滚动缓存把每层本地状态限制在窗口内：

$$M_{\text{SWA}} \approx 2sBLh_{\text{KV}}d_h \min(T, W). \quad (1.20)$$

代价是失去对窗口外原始位置的直接访问；信息只能通过后续状态间接传递，或者依赖额外的 global 分支。Mistral 7B 是 GQA 与滑动窗口结合的一个代表实例。[18]

动态稀疏 attention 每个 query 只选 Top-K 历史位置，首先减少的是读取和主 attention 计算。若下一个 query 可能选择别的位置，就仍然需要保留其余缓存。“这次不读”与“以后不存”是两件事。[7]

时间压缩则把多个历史位置合并成一个表示，真正减少缓存 entry 的数量。它会改变信息的表示方式，存在压缩损失，通常需要架构与训练适配。V4 的 CSA/HCA 属于这一路线。[8]

1.6.4 跨层共享：直接减少独立维护 global memory 的层数

Cross-Layer Attention、YOCO 等工作将一层构造的 KV 交给其他层读取，减少独立缓存份数。它们与 MQA/GQA 的 head 共享可以形成不同维度的组合。YOCO 还利用上下半网络的分工，让 prefill 得以提前结束。[9, 10]

但共享程度越高，就越需要考虑不同层是否仍能读取适合自己的历史表示。V4.1 的选择是**共享 global KV**，同时保留每层自己的 Q、SWA KV 和后续计算；CSA2 又将“共享缓存”和“共享索引结果”分开。[1, §2.3.1, pp. 10–11]

1.6.5 量化、分页、FlashAttention、offload：作用在不同层面

低比特 KV降低每个数的存储成本 s ，但还需要存 scale 等元数据，并控制量化误差。权重量化不自动等于 KV 量化：两者是不同张量、不同生命周期的优化。[1, §2.4.4, p. 14]

PagedAttention主要解决物理内存分配、碎片和可共享块的重复复制。它能显著提高已分配内存的利用率，却不会把某层一个独立 KV entry 的数学维度变小。[17]

FlashAttention优化 attention 的 IO 路径，避免在 HBM 中显式物化完整的 attention 分数矩阵；它可以保持精确 attention 语义。它处理的二次中间矩阵问题，与跨生成步骤保留的线性 KV cache 问题不同。[16]

Offload 与 persistent prefix cache把数据放到 CPU 内存、SSD 等更便宜的存储层，或跨请求保留可复用前缀。这改善容量与复用成本，但缓存字节仍然存在，并增加加载、迁移和失效管理问题。V4.1 专门讨论了 global KV 与 SWA 快照不同的生命周期。[1, §3.2.1, p. 19]

1.7 带着这张“乘法账单”进入 V4.1

对具有 global 与 local 双分支的模型，可写出一个更一般的逻辑存储记账式。设 $\mathcal{U}$ 是真正独立生成 global KV 的层集合，每个这样的层有时间压缩率 m_u ，每个 global entry 共占 D_u bytes；每层 local entry 占 D_ℓ^{local} bytes：

$$M_{\text{runtime}} \approx B \left[T \sum_{u \in \mathcal{U}} \frac{D_u}{m_u} + \sum_{\ell=1}^L \min(T, W_\ell) D_\ell^{\text{local}} \right]. \quad (1.21)$$

这里 D_u 应包含 Main KV、Indexer K 及量化 scale；式子不计权重、临时工作区、内存对齐和管理元数据。压缩块的尾部边界也被近似忽略。

这个式子把 V4.1 的设计清楚地拆开：**FP4 减小 D_u ；时间压缩增加 m_u ；跨层共享减少集合 $\mathcal{U}$ ；SWA 限制局部历史长度**。Top-K 改变的是访问选择，不会单独把整个 global 缓存中的 T 替换成 K 。长期 persistent cache 还要另算快照数量和保留策略。[1, §2.3、§2.4.4、§3.2.1]

接下来最重要的问题便不再是“能不能把每个数字再压小一点”，而是：**未来 token 究竟需要多少套历史表示？这些表示需要经过多深的计算才足够？**

第 2 章 整体图景：它重新划分了什么？

这份报告最值得关注的，不是“又把 MoE 做得更稀疏了一点”，而是它重新划分了三件事情：历史 token 需要经过多少计算才能形成记忆；这些记忆需要保存多少份；生成当前 token 时，需要读取哪些记忆、进行多少层计算。

我的理解是，DeepSeek-V4.1-Flash 的核心设计可以概括为：

用相对便宜的路径构造历史信息，用跨层共享的方式保存历史信息，再让当前生成位置经过完整的深层计算。

这也是 CED、CSA2、FP4 KV 和 SWA Bounded Replay 能够相互配合的原因。报告本身围绕长周期 agent 的 prefill 计算、KV 存储和缓存迁移带宽展开，而不只是优化单个 attention kernel。[1, §1, pp. 4–6]

2.1 它到底是一个什么模型？

原报告第 7 页的 Figure 3 是整篇报告最重要的一张图。语言主干一共 40 层，分成前面的 20 层 causal encoder 和后面的 20 层 decoder；所有层的 FFN 都是 MoE。前两层只有局部滑动窗口注意力，其余层都使用 CSA2。除此之外，还接入视觉编码器、Engram、Single-Pass mHC 和 DSpark。

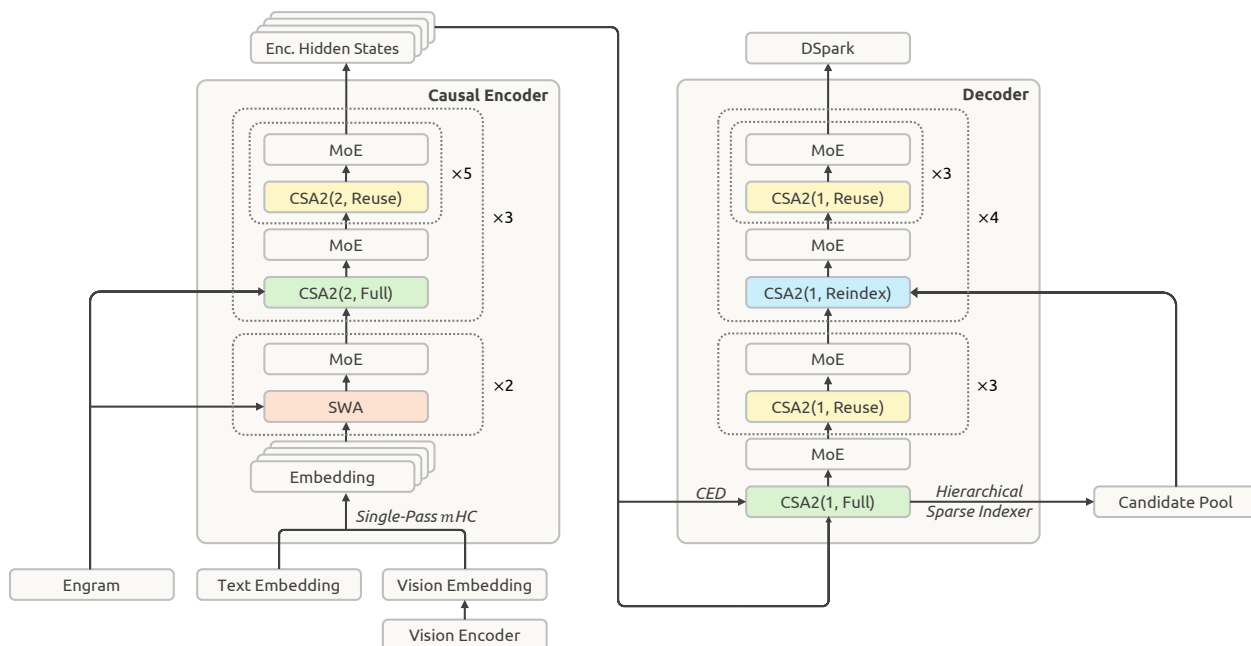


图 2.1 整体架构。摘自上传报告第 7 页 Figure 3，保留原图符号：CSA2(ratio, mode) 分别指定时间压缩率和层模式。[1]

表 2.1 V4.1-Flash 的基础配置

项目	配置
语言主干	40 层, hidden dimension 为 5120
Encoder / Decoder	各 20 层
主干参数	552B
Engram 参数	额外的 196B , 与主干分开统计
每层 MoE	384 个 routed experts, 另有 1 个 shared expert
每个 token 的专家选择	6 个 routed experts, 加 shared expert
Main attention	64 个 query heads, 每个 head 512 维
Sparse indexer	32 个 query heads, 每个 head 128 维
每次选取的 global entries	最多 512 个
局部 SWA 窗口	128 tokens

这些配置分别来自架构总览和原报告 §4.2.1。注意, **552B 不是把 196B Engram 包含进去后的总数**。[1, §2.1, p. 7; §4.2.1, pp. 21–22]

2.2 先区分三种“记忆”

Main KV 是真正参与主 attention 的历史表示。

Indexer K 是辅助检索的低维表示：先用它判断哪些历史位置值得看，再去读取对应的 Main KV。这两者合起来，是报告所说的 **global KV**。

此外, 每层还有自己独立的 **SWA KV**, 只保留最近一小段上下文, 负责局部的细粒度依赖。global KV 随上下文长度增长, 而固定窗口的 SWA KV 不随整个历史长度无限增长。[1, §1, p. 4]

因此, 一个很有用的理解方式是: **SWA 负责当前附近的精细处理; global attention 负责从整个历史中调取信息**。这不是强行套上的解释: 报告自己就提出, 可以把这类架构看成“以 SWA 为局部处理主干, 再用压缩后的全局上下文增强”。[1, §1, p. 4]

2.3 从 DeepSeek 之前的工作走到这里

这条演进线可以简化为:

$$\text{MLA} \longrightarrow \text{DSA} \longrightarrow \text{CSA/HCA} \longrightarrow \text{CSA2}. \quad (2.1)$$

这是一条便于理解设计重点的脉络, 并不表示后者只是把前者原封不动地叠加。

2.3.1 MLA：先缩小每个历史位置的表示

DeepSeek-V2 的 MLA 不直接保存所有 attention heads 展开后的 K、V，而是先生成共同的低维 latent。此处沿用前作的列向量记法：

$$c_t^{KV} = W^{DKV} h_t, \quad k_t^C = W^{UK} c_t^{KV}, \quad v_t^C = W^{UV} c_t^{KV}. \quad (2.2)$$

推理时主要缓存 latent，而不是展开后的多头 K/V；相关投影还可以吸收到 query、output 投影中。实际 MLA 另外缓存解耦的 RoPE key 分量，以解决位置变换与矩阵吸收的兼容问题。它主要压缩的是每个位置的通道/头维度，而不是历史位置的数量。[6, §2.1.2–2.1.3]

2.3.2 DSA：再减少每次读取多少历史位置

DeepSeek-V3.2 的 DSA 引入轻量 indexer，先选择 Top-K 历史位置，再让主 attention 只计算这些位置。它把“便宜地判断相关性”和“昂贵地读取、整合内容”分开了。[7, §2.1]

这里有一个贯穿全文的重要区别：只读取 Top-K，不等于只需要保存 Top-K。当前 query 没选中的历史位置，下一个 query 仍然可能需要，因此不能因为 attention 稀疏，就把其余历史缓存全部删除。

2.3.3 CSA/HCA：再减少历史表示的数量

V4 的 CSA 先把多个 token 的 KV 压成一个 entry，再做稀疏选择；HCA 则采用更重的压缩，然后对压缩后的条目做非稀疏 attention。V4-Flash 中两者的时间压缩率分别为 4 和 128。[8, §2.3、§4.2]

2.3.4 CSA2：进一步减少同一历史需要保存多少份

V4.1 的重点转向层维度共享：不同层不必各自产生和保存一套 global KV，也不必各自重新做一次完整索引。同时，它不再使用 CSA–HCA 混合结构，而是统一为 CSA2。[1, §1, p. 4; §2.3, pp. 9–11]

第 3 章 CED：为什么历史 token 可以少算一半层？

这是理解 **prefill 8B**、**decode 16B** 的核心。

3.1 普通 decoder-only 模型为什么不能随便跳过上半层？

先考虑一个普通的 40 层 Transformer。对历史位置 t ，第 ℓ 层的 KV 来自该层的 hidden state。沿用报告的层输入记法，可以写成：

$$K_t^{(\ell)} = H_t^{(\ell)} W_K^{(\ell)}, \quad V_t^{(\ell)} = H_t^{(\ell)} W_V^{(\ell)}. \quad (3.1)$$

因此，要得到第 35 层的历史 KV，就必须先让这个历史 token 经过前面的那些层，得到相应的 hidden state。

即使不需要 **prompt 中每个位置的最终 logits**，仍然需要其上层 KV，供未来生成的 token 使用。于是，常规模型的 prefill 仍然要把 prompt 跑过整个网络。第 1 章已经说明，这是逐层状态依赖带来的结果。CED 改的正是这个依赖关系。

3.2 关键改动：decoder 的 global KV 不再来自 decoder 自己

报告 §2.2 的公式是：

$$C_\ell = H_{L/2} W_\ell^{KV}, \quad Z_\ell = H_{L/2} W_\ell^Z, \quad \ell > L/2. \quad (3.2)$$

其中， C_ℓ 是用于形成 KV 的表示， Z_ℓ 是对应的压缩权重。对这个 40 层模型来说：

Decoder 的 global KV，由 encoder 最后一层 H_{20} 直接投影得到。

它不再要求先算出历史 token 在 decoder 第 25、30、35 层的 hidden state，才能得到这些层要使用的 global KV。[1, §2.2, p. 9, 式 (1)]

这带来一个根本性变化：**为了让历史 token 成为未来可读取的全局记忆，不再需要把它们逐个经过整个 decoder。**

这里先给出的是 CED 的一般表达。落实到本模型的 CSA2 配置，decoder 只有一个 Full Mode 层负责产生 global KV，后面的 decoder 层继续复用它；不是每层都真的保留一套不同的投影结果。[1, §2.3.1, p. 11; §4.2.1, p. 22]

3.3 为什么 prefill 可以跳过，decode 却不能跳过？

因为两个阶段的任务不同。对于 prompt 中绝大多数历史位置，prefill 的目的主要是：

$$\text{历史输入} \longrightarrow \text{未来可使用的缓存}. \quad (3.3)$$

而生成当前 token 时，还需要：

$$\begin{aligned} \text{当前表示} &\longrightarrow \text{逐层读取历史} \\ &\longrightarrow \text{逐层进行 MoE 计算} \\ &\longrightarrow \text{最终预测分布.} \end{aligned} \tag{3.4}$$

即使 decoder 的历史 KV 是共享的，当前 token 的 **query**、**attention output**、**MoE output** 仍然要逐层更新。所以当前生成位置仍然要走完前 20 层和后 20 层。

可以把一个普通生成步骤概括为：

$$x_t \longrightarrow H_t^{20} \longrightarrow \begin{cases} \text{补充 global memory,} \\ \text{继续经过 20 层 decoder} \longrightarrow H_t^{40} \longrightarrow p(x_{t+1}). \end{cases} \tag{3.5}$$

因此，CED 不是“输入阶段调用一个小模型，输出阶段切换成另一个大模型”，而是同一个模型的计算图允许历史位置提前完成缓存构建，而当前预测位置继续完成完整计算。[1, §2.1–2.2, pp. 8–9]

3.4 它与传统 encoder-decoder、YOCO 有什么区别？

这里的 encoder 是 **causal encoder**：不能看到未来。它也不是传统翻译模型那种“encoder 只处理源语言，decoder 只处理目标语言”的划分。整个语言模型仍然按因果顺序建模。[1, §2.1, p. 7]

其直接前作是 YOCO：下半部分 self-decoder 构造共享 KV，上半部分 cross-decoder 读取这些 KV，借此让 prefill 提前退出。[9]

CED 在此基础上的关键保留是：虽然 decoder 的 global KV 来自 encoder，但 decoder 每一层仍然保留自己生成的局部 SWA KV。

这使得局部信息能够经过更深的计算，而不是所有历史信息都只停留在 encoder 的表示深度。但代价是 prefill 不能真的把整个 decoder 完全省掉，还要解决局部缓存的初始化问题。[1, §2.2, p. 9]

这个问题稍后由 **Decoder SWA Bounded Replay** 解决，见第 7 章。

第 4 章 CSA2：究竟共享了什么，又保留了什么？

CED 解决的是“历史 KV 从哪里来”；CSA2 进一步解决“多少层需要各自保存、索引这些 KV”。

4.1 一次 CSA2 attention 的计算过程

先不考虑跨层共享，一个完整 CSA2 层大致需要做：

$$\begin{aligned} & \text{构造 Main KV} \longrightarrow \text{构造 Indexer K} \\ & \longrightarrow \text{Indexer 选出 Top-512} \\ & \longrightarrow \text{读取对应 Main KV.} \end{aligned} \tag{4.1}$$

然后，把选出的 global entries 与本层最近 128 个位置的 SWA KV 拼接起来，交给主 attention。

也就是说，长上下文下每个主 attention query 处理的是至多：

$$512 + 128 = 640 \tag{4.2}$$

个条目，而不是直接对整个百万 token 历史执行主 attention。这是根据报告配置得到的条目数量；其中 encoder 的一个 global entry 可以代表两个原始 token。边界处因果可见的条目不足时，实际数量更少。[1, §2.3, p. 10; §4.2.1, p. 22]

4.1.1 为什么有 64 个 query heads，缓存却不是 64 份？

V4 的相关前作已经采用 **Shared Key-Value MQA**：不同 query heads 共享同一组 KV 表示，同一个缓存 entry 同时作为主 attention 的 key/value 表示来源。它不是“64 个 query heads 就必须有 64 份独立 KV”。[8, §2.3.1]

在 V4.1 中，Main KV latent 为 512 维。因此，后面计算缓存时，**不能再乘一个 64，也不能按两套独立存储的 K 和 V 再乘一个 2**。报告的 512 维 latent 与缓存格式见 §2.4.4。[1, §2.4.4, p. 14]

4.2 Compressor：压缩的是内部表示，不是文本摘要

CSA2 的 encoder 使用时间压缩率 $m = 2$ ，decoder 使用 $m = 1$ 。

Encoder 大致每两个原始位置产生一个 global entry；Decoder 则每个原始位置保留一个 global entry，不进行时间维合并。这里的“压缩”不是先把文本摘要一下，而是对投影后的连续向量进行学习式聚合。[1, §4.2.1, p. 22]

结合 V4 的压缩公式，可以用下面这个省略位置变换、归一化细节的示意式理解：

$$c_j = h_j W_C, \quad z_j = h_j W_Z, \tag{4.3}$$

$$\bar{c}_b[r] = \sum_{j \in \text{block } b} \frac{\exp z_j[r]}{\sum_{u \in \text{block } b} \exp z_u[r]} c_j[r]. \tag{4.4}$$

其中 r 是通道索引。含义是：不同通道可以学习不同的聚合权重，不是机械地对两个 token 做平均。前作使用的是这种按通道的加权压缩。[8, §2.3]

V4.1 对 compressor 的简化主要有两点。

第一，去掉重叠压缩。原 CSA 的一个压缩 entry 涉及 $2m$ 个来源，相邻压缩 entry 的来源存在重叠；CSA2 不再保留这个重叠结构，也去掉了 compressor 内的绝对位置 embedding。

第二，Indexer K 直接从 Main KV 投影。不再从 hidden states 另起一条独立压缩路径来生成索引键。[1, §2.3, p. 10]

注意，去掉的是 **compressor 内的绝对位置 embedding**，不是去掉整个模型的位置机制；本模型仍然使用 RoPE。[1, §2.4.4, p. 14]

4.3 Indexer：先便宜地筛选，再进行主 attention

在前作 DSA 中，索引分数的原型是：

$$I_{t,s} = \sum_{j=1}^{H^I} w_{t,j}^I \text{ReLU}(q_{t,j}^I \cdot k_s^I). \quad (4.5)$$

然后按 $I_{t,s}$ 选出 Top-K 位置。它不需要像主 attention 一样完整地整合 value，只需要判断“哪些位置值得读”。[7, §2.1]

V4.1 的 indexer 配置是 32 个 128 维 query heads，明显小于主 attention 的 64 个 512 维 query heads；本报告重点改变的是索引键的来源、跨层复用和后续索引的搜索范围。[1, §2.3, p. 10; §4.2.1, p. 22]

关于“indexer 怎么学”的前作背景：V3.2 用主 attention 的聚合分布作为目标，通过 KL loss 训练 indexer。但不能把 V3.2 的 dense warm-up 流程原样套到 V4.1；V4.1 明确说自己从 64K 序列长度开始、直接以稀疏 attention 训练，没有 dense attention warm-up。[7, §2.1][1, §1, p. 6; §4.2.2, p. 22]

4.4 Full、Reindex、Reuse：三种模式的区别

原报告第 10 页 Figure 4 把三种模式画得很清楚。

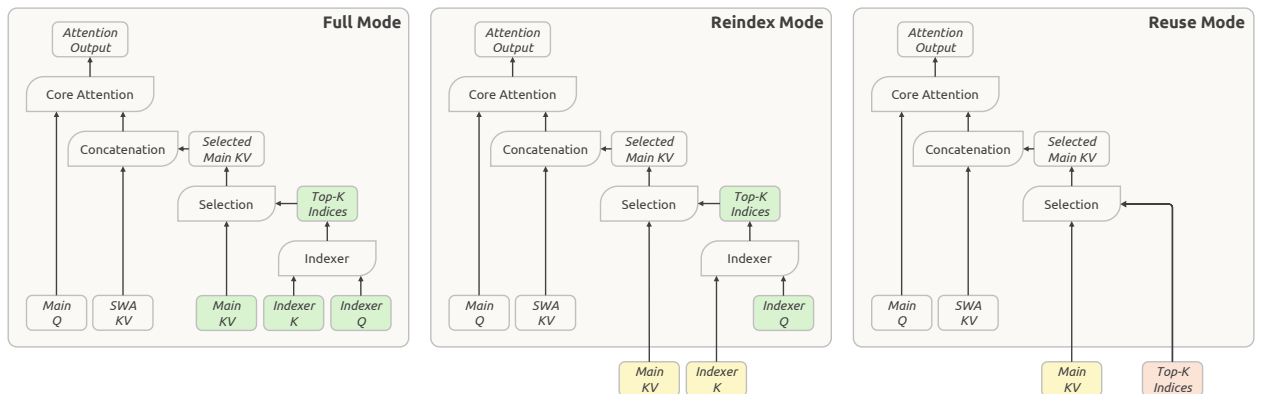


图 4.1 CSA2 的三种模式。摘自原报告第 10 页 Figure 4。绿色代表本层产生，黄色代表复用已有 KV，红色代表复用已有 Top-K indices。[1]

表 4.1 缓存共享与索引共享被分开控制

模式	Main KV、Indexer K	Top-K 位置	本层仍然计算
Full	本层产生	本层重新索引	Main Q、SWA KV、主 attention
Reindex	复用前面的 Full 层	用自己的 Indexer Q 重新选择	Main Q、SWA KV、主 attention
Reuse	复用前面的 Full 层	连选择结果也复用	Main Q、SWA KV、主 attention

模式是预先固定在各层上的架构配置，不是每个 token 动态决定本层要不要执行。动态变化的是各个 query 的索引结果。[1, §2.3.1, pp. 10–11]

最容易误解的是 Reuse：

复用 KV 和索引，不等于复用 attention output，更不等于跳过这一层。

假设第 ℓ 层复用了相同的历史条目集合 $\mathcal{S}_t$ ，它依然使用自己的 query：

$$o_t^{(\ell)} = \text{Attention}(q_t^{(\ell)}, C_{\mathcal{S}_t}, C_{\mathcal{S}_t}). \quad (4.6)$$

这里仅写出 global 部分，省略局部 KV 和位置变换。

由于 $q_t^{(\ell)}$ 不同，即使读取相同的历史条目，各层赋予它们的权重仍然可以不同，产生的输出也不同；之后还有各层自己的 MoE。报告明确保留了每层自己的 Q、SWA KV 和新的 attention output。[1, §2.3.1, p. 10]

所以它消除的是“重复保存历史”和“重复寻找历史位置”，而不是取消深层计算。

4.5 具体到 40 层，究竟有几套 global KV？

按从 1 开始的层编号整理，结构如下。这是将原报告 §4.2.1 的分组配置展开后的结果。[1, §4.2.1, pp. 21–22]

表 4.2 40 层的模式分配；层编号为 1-based

层范围	配置	时间压缩率
1–2	纯 SWA，不构造 global KV	不适用
3–8	1 个 Full + 5 个 Reuse	$m = 2$
9–14	1 个 Full + 5 个 Reuse	$m = 2$
15–20	1 个 Full + 5 个 Reuse	$m = 2$
21–24	1 个 Full + 3 个 Reuse	$m = 1$
25–28、29–32、33–36、37–40	每组 1 个 Reindex + 3 个 Reuse	$m = 1$

因此，整个主干有：

$$4 \text{ 个 Full} + 4 \text{ 个 Reindex} + 30 \text{ 个 Reuse} + 2 \text{ 个纯 SWA.} \quad (4.7)$$

真正独立产生 global KV 的只有 4 个位置。

其中三个 encoder Full 各保存约 $N/2$ 个 entries，一个 decoder Full 保存约 N 个 entries，所以总 global entry 数是：

$$3 \times \frac{N}{2} + N = 2.5N. \quad (4.8)$$

这是后面算出 890 bytes/token 的关键。这里不是“40 层只剩下 4 层”，而是：40 层仍然进行计算，但需要长期维护的 global memory 只有 4 组。

4.6 Hierarchical Sparse Indexer：限制后续索引的搜索范围

仅仅减少 indexer 的数量还不够。剩下的每个 indexer 如果仍然扫描百万个位置，开销依然会增长。

V4.1 的做法是：decoder 第一个 Full indexer 完成一次全范围搜索，后面的 Reindex 层只在它圈出的候选池中重新选择。这就是原报告第 11 页 Figure 5。[1, §2.3.2, pp. 11–12]

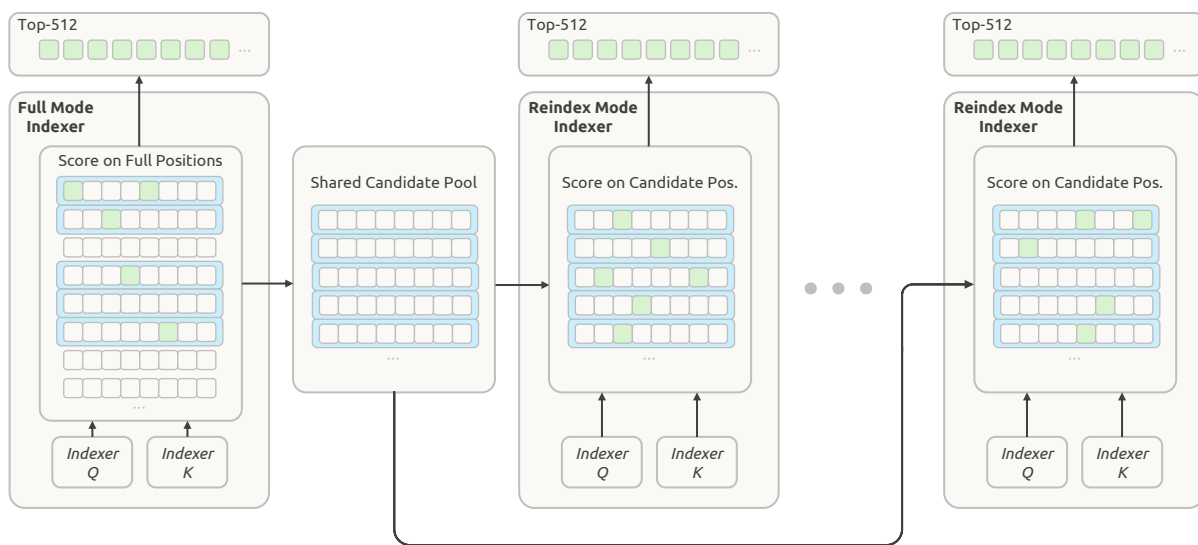


图 4.2 分层稀疏索引。摘自原报告第 11 页 Figure 5。第一个 Full indexer 构造候选池，后续 Reindex 层仅在池内选出各自的 Top-512。[1]

具体过程是：第一个 Full indexer 对所有因果可见的位置打分，既选出自己要用的 Top-512，也按连续位置分块。每块包含 8 个位置，块分数取其中最高的索引分数。选择最多 2048 个块，得到：

$$2048 \times 8 = 16384 \quad (4.9)$$

个候选位置。后面的四个 Reindex 层，只在这最多 16384 个位置中重新选自己的 Top-512，而不是每次再扫整个百万 token 历史。[1, §2.3.2, p. 12; §4.2.1, p. 22]

几个细节很重要。

候选池不是最终读取集合。候选池最多 16384，主 attention 最终仍只读选出的 512 个 global entries。

共享候选池不意味着共享最终选择。不同 Reindex 层的 query 不同，可以在候选池里选出不同的 512 个位置。

这个池是针对当前 query 构建的。不是整段对话永远固定一份候选池。

第一个 Full indexer 仍然扫描全部历史。所以整个 decode 计算并没有在数学上变成严格的 $O(1)$ 。报告说的是后续 indexer 的搜索成本被限制住了。[1, §2.3.2, p. 12]

例如，只数长上下文下“索引器需要评分的 entry 数”，忽略头维度等常数，依据本模型的布局可以估算为：

$$3 \times \frac{N}{2} + N + 4 \times 16384. \quad (4.10)$$

第一项来自 encoder 的三个 Full，随后是 decoder 的一个 Full，最后是四个受限的 Reindex。它仍然包含 N 项，但不再让大量深层 indexer 重复全范围搜索。

代价也很明确：如果第一个 decoder indexer 没把关键位置纳入候选池，后面的 Reindex 就无法重新发现它。这也是报告最后承认仍需继续压力测试的 sparse selection 风险之一。[1, §6, p. 37]

补充一个训练口径：hierarchical candidate restriction 在 post-training 中引入，并在训练、推理时采用相同的搜索限制，令深层 indexer 适应部署时的候选范围。[1, §2.3.2, p. 12]

4.7 低秩 Query 与分组 Output Projection

这部分不是最醒目的新贡献，但它解释了为什么 attention 的投影计算没有随着大 head dimension 一起变得过重。

V4.1 的 query 压缩维度是 1280，可以把路径理解为：

$$5120 \longrightarrow 1280 \longrightarrow 64 \times 512. \quad (4.11)$$

Attention 输出则分成 8 组，每组的 8 个 heads 先从 $8 \times 512 = 4096$ 维投影到 1024 维，再拼接并投影回主干维度：

$$8 \times 4096 \longrightarrow 8 \times 1024 \longrightarrow 5120. \quad (4.12)$$

低秩 query、分组 output projection 的基本做法来自 V4，本模型沿用相应结构并采用上述配置。它们主要减少投影计算，不是通过压缩 Q 来减少 KV cache——历史 Q 本来就不需要长期缓存。[8, §2.3][1, §4.2.1, p. 22]

第 5 章 三个辅助架构：mHC、Engram 与 DSpark

这三个组件的作用完全不同，最好不要统一归入“提升模型速度”的同一个篮子。Single-Pass mHC 优化的是残差流的表达与数据搬运，Engram 增加的是稀疏访问的参数化记忆，DSpark 则减少自回归生成的串行开销。[1, §2.4.1–§2.4.3, pp. 12–14]

5.1 Single-Pass mHC：多条残差通路，不多搬运数据

5.1.1 从普通 residual connection 开始

普通残差连接是：

$$x_{l+1} = x_l + F_l(x_l). \quad (5.1)$$

mHC 则让每个 token 在 block 之间保留 n 条 residual streams：

$$X_l \in \mathbb{R}^{n \times d}. \quad (5.2)$$

本模型 $n = 4$ 。但这并不意味着每个 attention 或 expert 的 hidden dimension 都变成四倍。重计算模块 F_l 接收到的是由多条流混合得到的单个 d 维输入。[1, §2.4.1, p. 12; §4.2.1, p. 22]

原始更新式为：

$$X_{l+1} = B_l X_l + C_l F_l(A_l X_l), \quad (A_l, B_l, C_l) = \mathcal{H}(X_l). \quad (5.3)$$

三个系数分别承担不同职责。

$A_l \in \mathbb{R}^{1 \times n}$ 把多条流混成 block 输入； $B_l \in \mathbb{R}^{n \times n}$ 让 residual streams 之间相互混合； $C_l \in \mathbb{R}^{n \times 1}$ 把 block 输出分配回多条 residual streams。

这些系数由当前 token 的状态预测，因此不是所有输入都使用同一个固定混合矩阵。系数预测器 $\mathcal{H}$ 包含归一化和投影。[1, §2.4.1, p. 12]

5.1.2 “manifold-constrained” 约束的是什么？

前作 mHC 将残差混合矩阵 B_l 约束为近似双随机矩阵：

$$(B_l)_{ij} \geq 0, \quad B_l \mathbf{1} = \mathbf{1}, \quad \mathbf{1}^\top B_l = \mathbf{1}^\top. \quad (5.4)$$

这样，残差混合是不同流的凸组合，且该映射的谱范数不超过 1；连续相乘也保留双随机结构。约束针对的是残差混合通路，不能据此声称整个带非线性模块的网络绝不会出现信号放大。[11]

5.1.3 为什么还需要 Single-Pass？

原实现存在一个数据依赖：先得到 X_l ，再根据完整的 X_l 算出 A_l ，最后才能计算 $A_l X_l$ 。其中系数预测涉及跨 hidden dimension 的归约，导致 residual 数据需要被重复读取，妨碍一次性融合。[1, §2.4.1, pp. 12–13]

V4.1 将输入混合系数错开一个 block:

$$X_{l+1} = B_l X_l + C_l F_l(A_{l-1} X_l). \quad (5.5)$$

即当前 block 使用前一个 block 产生的混合系数。

这里是同一个 token 沿网络深度使用前一个 block 的系数，不是使用前一个时间位置的系数。

这样，处理当前 residual 的每个 tile 时，输入混合已经可以立即执行，无须等待当前 block 的系数归约完成。报告称这种错位只带来很小的性能影响。[1, §2.4.1, p. 13]

配合 Mega-mHC kernel, residual update、input mixing、coefficient prediction, 以及 pre-norm、FP8 conversion 可以融合。报告给出的相关 activation traffic 从:

$$(4n + 4)d \longrightarrow (2n + 2)d. \quad (5.6)$$

当 $n = 4$ 时，就是:

$$20d \longrightarrow 10d. \quad (5.7)$$

残差数据实现一次读取、一次写回，达到报告讨论的理想搬运下界。[1, §2.4.1, p. 13]

这是一半的相关 activation 搬运量，不是整个模型 FLOPs 或延迟直接减半。

5.2 Engram：用查表扩展容量，而非让所有知识都经过专家计算

MoE 扩展的是“可选择的计算模块”；Engram 扩展的是“可稀疏访问的参数化记忆”。报告把其目标概括为将 **memorization** 与 **computation** 解耦。V4.1 使用两个 Engram 模块，总共 196B 参数。[1, §2.4.2, p. 13]

5.2.1 它怎么查表？

原始 Engram 的路径是：先将 token ID 映射为规范化 ID，再提取当前位置的后缀 N-gram；对每个 N-gram 使用多个 hash head，分别查不同 embedding tables，并拼接取出的向量。Tokenizer compression 压缩的是查表所用的 ID 空间，不是缩短输入 token 序列。多头 hashing 用于降低单次哈希碰撞的影响。[12]

V4.1 使用的 N-gram orders 是:

$$\{2, 3, 4\}. \quad (5.8)$$

每个 order 有 8 个 hash heads，总 embedding dimension 为 2048；每张表约 16M 行，大小选择为不同质数。两个模块放在 zero-indexed 的第 1、14 层，即通常从 1 计数的第 2、15 层。[1, §2.4.2, p. 13]

因此，某个位置只会读取少数 embedding rows，而不是对整个 196B 参数做一次 dense 运算。

5.2.2 为什么查出来的向量还需要 gating?

固定 N-gram 查到的是静态表示，可能存在多义性或碰撞。原 Engram 用当前 hidden state 与查得表示产生的 key 计算 gate:

$$k_t = W_K e_t, \quad v_t = W_V e_t, \quad (5.9)$$

$$\alpha_t = \sigma \left(\frac{\text{RMSNorm}(h_t)^\top \text{RMSNorm}(k_t)}{\sqrt{d}} \right). \quad (5.10)$$

然后注入 $\alpha_t v_t$ 。在多 residual branches 下，共享表与 value 投影，但使用分支特定的 key 投影形成不同 gates。[12]

所以它不是“查到什么就无条件塞进去”，而是让动态上下文决定静态记忆的使用程度。

5.2.3 为什么它适合放在主存?

关键在于：查表地址只依赖已经知道的 token IDs，不依赖还没有算出来的深层 hidden state。

因此，可以提前计算地址，从 host memory 预取 embedding，通过后台 RDMA 传输，并与前面 Transformer block 的计算重叠。V4.1 的 embedding tables 和 key/value projections 都使用 FP8。[1, §2.4.2, p. 13]

V4.1 还删掉了原 Engram 的短 causal convolution，因为其收益不足以抵消推理实现复杂度，并修改了大表的优化器。[1, §2.4.2, p. 13]

Engram 不是会话缓存。它是跨请求共享的、训练得到的模型参数，不是某个用户会话的 KV cache，也不是外部 RAG 文档库。它增加的是模型容量，并不能直接用来解释会话 KV 为什么从 1 变成 1/4。

5.3 DSpark：一次提出多个候选，但不盲目验证所有候选

DSpark 解决的是自回归生成的串行瓶颈。

V4.1 中的 drafter 有 3 个 Transformer blocks，滑动窗口为 128；一次前向并行产生 5 个 draft positions 的 base logits。之后用轻量 Markov head 补上候选 token 之间的依赖，再由 confidence head 和 scheduler 决定验证长度。[1, §2.4.3, pp. 13–14]

5.3.1 为什么不能直接独立预测五个位置?

因为独立预测容易让不同位置选择互不一致的续写。

DSpark 的折中是：昂贵的表示计算并行完成，便宜的条件修正顺序进行。

在前作的 Markov head 中，位置 k 的 logits 可写成：

$$\ell_k(v) = U_k(v) + B(x_{k-1}, v), \quad (5.11)$$

$$B = W_1 W_2, \quad B(x_{k-1}, \cdot) = W_1[x_{k-1}]W_2. \quad (5.12)$$

U_k 是并行产生的 base logits，后一个项是由前一个已采样 token 决定的低秩转移修正。它只保留很轻的串行计算，而不必为每个 draft token 重新跑一遍 drafter 的 Transformer。[13]

5.3.2 Confidence scheduling 为什么有用?

即使已经生成了五个候选，也不代表验证五个最划算。

设 a_k 是“前面候选都通过后，第 k 个也通过”的条件概率，则前 k 个全部通过的估计概率为：

$$S_k = \prod_{j=1}^k a_j. \quad (5.13)$$

由此，验证前 γ 个候选时，接受前缀长度的期望为：

$$\mathbb{E}[A_\gamma] = \sum_{k=1}^{\gamma} S_k. \quad (5.14)$$

这是根据条件概率和非负整数变量的尾和公式得到的**推导**，只统计接受的 draft 前缀。越靠后的候选，即使本身不是很差，也可能因为前面某个位置被拒绝而完全失去价值。

V4.1 的 scheduler 将这些估计与**实测的 engine throughput curves**、**当前系统负载**结合，为不同请求动态选择验证长度，目标是系统整体 token throughput，而非单纯追求最长 draft。[1, §2.4.3, p. 14]

主模型仍然负责验证候选；confidence head 是用来分配验证预算的，不是代替主模型批准输出。并行验证后，只保留通过的前缀，拒绝点后的候选不能直接沿用。[13]

因此：

$$\begin{aligned} \text{一次提出 5 个候选} &\neq \text{每轮一定接受 5 个} \\ &\neq \text{固定获得 5 倍加速.} \end{aligned} \quad (5.15)$$

5.3.3 它和以前的 MTP 有什么训练区别?

V4.1 不在 backbone pretraining 中联合训练原来的 MTP。

先完成 backbone pretraining，再冻结 backbone、单独训练 DSpark；后续 post-training 中继续同步训练 DSpark，但不把 DSpark objective 的梯度传回 backbone。这样既能跟随策略变化，也能用于加速 serving、RL rollout 和 OPD rollout。[1, §2.4.3, p. 14]

第 6 章 FP4 KV：低精度存储，不等于全部 attention 都用 FP4 算

这一章直接关系到 KV 缩减。要理解报告中的 FP4，必须先区分存储格式和实际执行矩阵乘法的数值格式。

6.1 两种 FP4，用途不同

报告区分了两条路径。

Indexer Q/K 使用 MXFP4。它既可以减小 indexer cache，又能用于加速索引计算。

Main KV 使用另一种约四位格式。它主要用于缩小缓存，读取之后先反量化，再进入 attention 计算。因此，该缓存格式不要求硬件原生支持相应格式的矩阵乘法。[1, §2.4.4, p. 14]

所以不能从“FP4 Main KV”推断：“整个主 attention 的矩阵乘法都变成 FP4，算力自然四倍。”报告明确说了 Main KV 在这里主要是 **storage optimization**。

6.2 Main KV 的具体格式与数值范围

Main KV 使用：

$$\text{E2M1 数据} + \text{每 16 个通道一个 E4M3 scale.} \quad (6.1)$$

它借鉴 NVFP4，但省去了第二级 global scale。可以把一个值的重构理解为：

$$\hat{x} = s_{\text{block}} x_{\text{FP4}}. \quad (6.2)$$

每个值主体占 4 bits，但 scale 本身也占空间，因此有效存储成本高于恰好 4 bits/value。[1, §2.4.4, p. 14]

6.2.1 为什么能省掉 global scale?

报告给出了一个数值范围上的论证。

Main KV latent 是 512 维，训练后的 RMSNorm 最大权重幅度约为 1，所以归一化后的 latent 的 L_2 范数上界约为：

$$\sqrt{512} \approx 22.6. \quad (6.3)$$

RoPE 保持这个范数，因此旋转后单个通道的绝对值也不会超过这个界。实际训练中观察到的最大值约为 10。

而该格式的最大可表示幅度约为：

$$448 \times 6 = 2688. \quad (6.4)$$

所以，省去第二级 scale 不会造成动态范围不够的问题。[1, §2.4.4, p. 14]

当然，**范围够大不等于量化误差为零**。报告进一步通过 post-training 中的 QAT 让模型适应这种缓存格式，并在 RoPE 之后量化，避免 decode 时增加额外位置变换开销。非 RoPE 与 RoPE 分量使用相同的量化格式。[1, §2.4.4, p. 14]

6.3 为什么 SWA 不也压成 FP4?

因为报告发现 SWA KV 对量化更敏感，所以仍然保留 FP8。[1, §2.4.4, p. 14]

大量、长期增长的 global KV 用 FP4；
小量、局部精细处理的 SWA KV 保留 FP8。

这不是全模型一刀切的最低精度，而是按照缓存角色分配精度。后面复原 890 bytes/token 时，必须把 FP4 主体和每块 scale 都计入，不能只做“维度乘 0.5 bytes”的粗略乘法。

第 7 章 SWA Bounded Replay：用近似重计算替代长期局部缓存

这项设计同时支撑两件事：CED 的低成本 prefill，以及 persistent KV 的 1/8 缩减。[1, §3.2.1–§3.2.2, pp. 19–20]

7.1 128-token 窗口，为什么恢复时可能需要重算更多？

因为 SWA 的依赖会沿深度累积。

第一层最近位置的表示依赖前面一个窗口；第二层又依赖这些已经聚合了更早信息的表示；层层叠加，理论感受野大致增长到：

$$L \times W. \quad (7.1)$$

因此，报告指出，精确恢复 L 层的 SWA KV，需要 replay 大约 LW 个 token。对 20 层 decoder、128 窗口来说，就是约：

$$20 \times 128 = 2560 \quad (7.2)$$

个 prompt tokens，而不是只处理最后 128 个。这里沿用报告的感受野规模记法，边界位置的精确计数取决于窗口定义。[1, §2.2, p. 9; §3.2.2, p. 20]

对于“缓存了一大段前缀，这一轮只新增很短输入”的 agent 请求，额外重算 2560 个位置就可能很不划算。

7.2 Bounded Replay 做了什么近似？

它只 replay 最近 $W = 128$ 个 token，并把局部 attention 截断在 replay 区间内。

若 replay 从位置 s 开始，则位置 i 的局部注意力范围变成：

$$[\max(s, i - W + 1), i]. \quad (7.3)$$

这意味着 replay 开头的一些位置，看不到原本完整执行时可以通过局部通路读到的更早状态。所以恢复出的 SWA KV 不是数学上等价的状态。报告接受这一近似，并报告其评估中能力损失很小。[1, §3.2.2, p. 20]

7.3 Encoder Bounded Replay：全局缓存命中，局部缓存缺失

当 global prefix cache 命中、但 encoder 的 SWA cache 没命中时，重跑缓存前缀的最后 128 个 token，并处理新增 suffix。

对于重跑的前缀，只重建 SWA KV，直接复用原来的 global KV，不重新计算或覆盖已缓存的 global KV。新 suffix 则生成自己的 global KV 和 SWA KV。[1, §3.2.2, p. 20]

这使得长期 prefix cache 不再必须依赖完整的 SWA 快照。由于重建状态是近似的，新 suffix 的 global KV 和 SWA KV 也可能依赖具体缓存命中边界，不同恢复位置下并非数学上完全一致。报告明确承认了这一点。[1, §3.2.2, p. 20]

7.4 Decoder Bounded Replay: 让 prefill 提前结束成为可用路径

每次 prefill，取 prompt 最后 128 个位置的 encoder outputs，让它们经过 20 层 decoder，构造生成初期要使用的 decoder SWA KV。

这些 decoder SWA KV 只用于当前 decode，不进入长期 prefix cache。报告还在 post-training 中模拟了这种 replay，使训练分布适应部署时的近似。[1, §3.2.2, p. 20]

因此，长 prompt 的主干计算量可以用下面的层—token 计数理解：

$$\underbrace{20N}_{\text{全部 prompt 经过 encoder}} + \underbrace{20W}_{\text{最后一个窗口经过 decoder}}, \quad (7.4)$$

而不是：

$$40N. \quad (7.5)$$

更一般地，报告的这一执行量记账是：

$$\frac{L}{2}N + \frac{L}{2}W \approx \frac{L}{2}N, \quad N \gg W. \quad (7.6)$$

这正是报告“prefill 几乎减半”的基础。[1, §2.2, p. 9]

这个计数描述主要网络层的执行量，**不能据此忽略 indexer 扫描等工作**，宣称整个算法所有计算都严格变成线性的 $O(N)$ 。同样，节约一半层—token 执行量也不保证所有硬件、所有长度下的端到端 prefill 延迟恰好减半。

第 8 章 其余组件：MoE、多模态、优化器与系统实现

8.1 MoE：大容量低激活的基础，但不是 8B/16B 差异的来源

DeepSeekMoE 的前作思路包括细粒度专家划分和 shared expert：用更多较小的专家获得更灵活的组合，并让 shared expert 承担共通能力，减少 routed experts 之间重复学习。[14]

V4.1 每层有 384 个 routed experts，选 6 个，加上 1 个 shared expert。忽略归一化等细节，可以写成：

$$y = E_{\text{shared}}(x) + \sum_{i \in \text{Top6}(x)} g_i(x) E_i(x). \quad (8.1)$$

每个专家的 intermediate dimension 为 2304，采用带阈值 10 clamping 的 SwiGLU。[1, §4.2.1, p. 22]

报告没有说 prefill 只选 3 个专家、decode 才选 6 个。8B/16B 的差别来自阶段执行路径，而不是减半 top-k。

8.1.1 多模态负载均衡

图像 token 和文本 token 的专家偏好可能不同。只看总负载，可能掩盖某一种模态内部的失衡。

V4.1 为图像、文本分别维护 expert correction biases；选择专家时用对应模态的 bias，但专家输出的混合权重仍使用原始 routing scores。两套 biases 按各自模态的负载独立更新。[1, §2.1.1, p. 8]

顺带注意，“auxiliary-loss-free”并不意味着这里完全不存在任何 balancing loss：训练配置仍保留一个权重为 10^{-4} 的小型 sequence-level balance loss，防止单序列出现极端失衡。[1, §4.2.2, p. 22]

8.2 多模态输入：视觉 token 数量也在压缩

视觉部分使用自训练的 DeepSeek-ViT：32 层、hidden dimension 1024、16 个 heads、patch size 14。它采用 2D-RoPE 适应不同分辨率，并使用 RMSNorm、SwiGLU 等设计。Patch embedding 使用线性投影而非卷积，以适配 Muon 优化器。[1, §2.1.1, p. 8；§4.2.1, p. 22]

视觉特征进入语言模型之前，先执行 3×3 pixel-unshuffle：

$$H \times W \times C \longrightarrow \frac{H}{3} \times \frac{W}{3} \times 9C. \quad (8.2)$$

它先把相邻空间位置的信息重新排列到通道维，再由 MLP projector 映射到语言模型维度。因此，进入 LLM 的视觉 token 数减少为原来的 $1/9$ 。这里的 H, W 是视觉网格尺寸，不是语言模型层数或 SWA 窗口。[1, §2.1.1, p. 8]

按报告配置计算，一个 1344×1344 图像对应：

$$96 \times 96 = 9216 \quad (8.3)$$

个 14-pixel patches；经过 3×3 重排后，对应：

$$32 \times 32 = 1024 \quad (8.4)$$

个 LLM 视觉位置。这压缩的是**进入语言主干的序列长度**；不能说 ViT 自身所有计算也自动减少为九分之一。

8.3 优化器：主要是在降低训练这些结构的成本

这部分不是直接的 decode 算子优化，但对于能否扩展到这种规模很重要。

主干线性矩阵主要使用 Muon；Query/Key 矩阵采用 head-wise Muon，让不同 attention heads 分别进行矩阵更新，而不是共享同一个整体预条件处理。[1, §2.5, pp. 14–15]

Engram、token embedding 和 prediction head 则采用 momentum update 加 Sinkhorn balancing，对更新矩阵的行、列尺度进行平衡。报告强调，该方法只需要 momentum buffer，避免为巨大表格承担 Adam 式额外 optimizer state 的开销。归一化参数、bias 等非矩阵参数仍使用 AdamW。[1, §2.5, pp. 15–16]

这里的 Sinkhorn 与 mHC 中的 Sinkhorn 不要混淆：**mHC 在约束 residual mixing matrix；优化器在平衡参数更新矩阵的行列 RMS。**[1, §2.5, p. 15]

8.4 系统实现：把结构节省兑现成实际速度

CSA2 的跨层共享还要求训练系统维护正确的共享状态和梯度。报告通过 shadow indexers、pipeline payload 和 microbatch 级生命周期管理，让共享组件跨 pipeline stages 时仍然保持单一逻辑所有者和正确梯度汇总。[1, §3.1.2, pp. 17–18]

具体而言，shadow indexers 在参与阶段保留轻量执行副本，但优化和 checkpoint 由单一逻辑所有者负责；跨阶段传递共享表示、索引和梯度；共享状态在最后一个 consumer 完成后及时释放。不能仅靠把一个 Python 模块引用复制到不同 pipeline stages，就自动获得这些语义。最后一句是对上述系统需求的实现层解释。[1, §3.1.2, pp. 17–18]

推理侧则通过 kernel fusion，把复杂结构压进很少的执行步骤。报告称 Reuse Mode 层只需要：

$$15 \text{ 个 prefill kernels}, \quad 11 \text{ 个 decode kernels.} \quad (8.5)$$

相关融合包括 FlashMLA、Mega-Gate、Mega-mHC、Mega-MoE，以及 TopK 等路径。[1, §3.2, pp. 18–19]

部署还采用 EPD disaggregation，让 **vision encoding、prefill、decode** 独立扩展和重叠执行。这里 EPD 的 Encoder 指视觉编码阶段，不能直接理解为“把 CED 的前 20 层全部单独拆成一个服务”。[1, §3.2, p. 19]

第 9 章 三个核心问题：加速、激活参数与 KV 缩减

9.1 问题一：它怎么大幅提高 inference 速度？

不是靠一个单独的技巧，而是分别优化不同瓶颈。

表 9.1 结构变化与主要受益场景

机制	主要减少什么	最直接影响的场景
CED + Decoder Bounded Replay	历史 prompt 的深层计算	长输入、缓存未命中的 prefill
稀疏主 attention	每层真正读取和整合的历史条目	长上下文 decode
Index reuse + hierarchical indexer	重复检索历史的计算	超长上下文 decode
跨层 KV 共享 + FP4	缓存容量与相应数据搬运	长上下文并发、缓存加载与迁移
Single-Pass mHC + kernel fusion	Activation 搬运、kernel 调用开销	高效低延迟执行
DSpark	每个最终输出 token 的串行生成开销	自回归 decode

这些机制分别对应报告的 CED/CSA2、mHC、DSpark 和推理系统设计。表格是对这些机制的归纳，不是独立的吞吐 benchmark。[1, §2.2–§2.4, pp. 9–14; §3.2, pp. 18–20]

9.1.1 为什么这些设计尤其适合 agent？

因为 agent 经常把工具结果、代码、日志重新作为输入，并反复恢复已有前缀。

于是，重要的成本不只是“生成一个 token 的乘法次数”，还包括：**新输入的 prefill、历史缓存的存储与读取、跨设备或存储层迁移、长会话下的并发容量**。报告正是把这些作为主要优化对象。[1, §1, p. 4; §2.2, p. 9]

9.1.2 但它是否在所有场景都更快？

不能这样下结论。

一个很关键的对照是：**V4-Flash decode 激活 13B 参数，而 V4.1-Flash decode 激活 16B**。新模型不是简单地把每个生成步骤都做小了；它是让更强、更大的主干在长上下文下避免大量随历史增长的额外成本。[1, Table 1, p. 24]

原报告第 5 页 Figure 2 给出的强结论是：上下文从 4K 扩大到 1M，即增长 256 倍，V4.1 的单 token decode FLOPs 只增加约 25%。但图中 FLOPs 按 BF16、FP8、FP4 分别用 1, 0.5, 0.25 加权，这不是实测 token/s 或 TPOT 曲线。[1, Figure 2 与 §1, p. 5]

因此，严谨的表述是：

它大幅改善了长上下文、输入密集型服务的计算与存储成本；实际延迟和吞吐提升，还取决于上下文、batch、缓存命中率、硬件、kernel，以及 DSpark 的接受率。

不能把“KV 变成 1/4”直接翻译成“decode 快四倍”，也不能把各模块的理论收益直接相乘。

9.2 问题二：为什么 decode 和 prefill 的激活参数可以不一样？

因为激活参数取决于实际执行的计算路径，而不仅仅取决于模型总参数。

对长 prompt：

大多数历史位置：20 层 encoder, (9.1)

最后 128 个位置：再经过 20 层 decoder. (9.2)

而正常生成当前 token 时：

当前位置：20 层 encoder + 20 层 decoder. (9.3)

这对应报告给出的 prefill 8B、decode 16B 规模口径。[1, §2.2, p. 9; §3.2.2, p. 20; §4.2.1, p. 22]

用层—token 数量计算，长 prompt 的相对工作量约为：

$$\frac{20N + 20W}{40N} = \frac{1}{2} + \frac{W}{2N}. \quad (9.4)$$

例如 $N = 8192, W = 128$ ：

$$\frac{1}{2} + \frac{128}{2 \times 8192} = 0.5078125. \quad (9.5)$$

也就是在这个简化计数下约为完整 40 层 prefill 的 50.8%。

但要加三个限定。

不是专家 top-k 减半。执行到的 MoE 层仍然选 6 个 routed experts，加 shared expert。

不是每个 prompt 位置都严格只激活 8B。尾部 replay 的位置仍要经过 decoder，另外也有缓存投影等工作；短输入时，这些边界成本占比更大。

不是训练也自动只算半个网络。当训练目标需要每个位置的完整预测时，仍然需要对应的 decoder 计算；prefill 的提前结束来自“历史位置只需要构建缓存”的任务差异。

前两点由报告配置和 replay 路径直接支持，第三点是上述计算依赖关系的推论。[1, §2.2, p. 9; §4.2.1, p. 22]

最关键的一句话：它让“构造历史记忆的深度”和“生成当前答案的深度”可以不同。

9.3 问题三：为什么 KV 能缩这么多，究竟是 1/4 还是 1/8？

两者都对，但指的不是同一类缓存。

9.3.1 先精确复原：890 bytes/token 是怎么来的？

下面是根据报告层数、维度、精度配置所做的**存储计算**，忽略边界残留、内存对齐和管理元数据。

第一笔：一个 Main KV entry 占多少？

512 维，每个值 4 bits：

$$512 \times \frac{4}{8} = 256 \text{ bytes.} \quad (9.6)$$

每 16 个通道一个 1-byte E4M3 scale：

$$\frac{512}{16} \times 1 = 32 \text{ bytes.} \quad (9.7)$$

合计：

$$B_{\text{main}} = 256 + 32 = 288 \text{ bytes.} \quad (9.8)$$

Main KV 的维度与格式来自原报告 §2.4.4。[1, p. 14]

第二笔：一个 Indexer K entry 占多少？

Indexer K 为 128 维，使用 MXFP4。该格式每 32 个数共享一个 E8M0 scale。[1, §2.4.4, p. 14; §4.2.1, p. 22][15]

因此：

$$128 \times \frac{4}{8} = 64 \text{ bytes,} \quad (9.9)$$

$$\frac{128}{32} \times 1 = 4 \text{ bytes.} \quad (9.10)$$

合计：

$$B_{\text{index}} = 64 + 4 = 68 \text{ bytes.} \quad (9.11)$$

一组 global entry 的主表示和索引表示总共：

$$288 + 68 = 356 \text{ bytes.} \quad (9.12)$$

第三笔：每个原始 token 对应多少组 global entries？

前面已经算过：

$$3 \times \frac{1}{2} + 1 = 2.5. \quad (9.13)$$

也就是三个压缩率为 2 的 encoder global caches，加一个压缩率为 1 的 decoder global cache。[1, §4.2.1, pp. 21–22]

最终：

$$\begin{aligned} B_{\text{global/token}} &= \left(3 \times \frac{1}{2} + 1\right) (288 + 68) \\ &= 2.5 \times 356 = 890 \text{ bytes.} \end{aligned} \quad (9.14)$$

这正好复原了报告中的 890 bytes/token。

所以这个数字同时依赖：

$$\begin{aligned} & \text{共享的低维 KV 表示} \times \text{时间维压缩} \\ & \times \text{跨层共享} \times \text{低精度存储}. \end{aligned} \quad (9.15)$$

这里的乘法表达强调各优化轴共同决定存储规模，并非把未定义的压缩率直接相乘。它不是单靠 FP4，也不是单靠稀疏 Top-K。不能因为有 64 个 query heads 就再乘 64，也不能对共享 Main KV latent 再按独立 K/V 乘 2。

9.3.2 这个 890 包含什么，不包含什么？

它是整个主干所有独立 global caches 合计后，摊到每个原始 token 上的成本，不是“每层每 token 890 bytes”。

它包含 Main KV 和 Indexer K。

它不代表整次推理峰值显存：模型权重、Engram、局部 SWA、临时激活、索引结果、分配器和系统管理开销不能全部塞进这个数字。报告明确区分 global KV、SWA KV 和 persistent cache。[1, §1, p. 4]

例如按十进制一百万 token 计算：

$$10^6 \times 890 = 890 \text{ MB}. \quad (9.16)$$

这是约 0.89 GB 的逻辑 global KV 数据，不是说运行这个模型总共只需要 0.89 GB 显存。

9.3.3 为什么说是 V4-Flash 的 1/4？

原报告第 1 页 Figure 1(b) 标出的 V4-Flash global KV 是 3514 bytes/token，V4.1-Flash 是 890 bytes/token。[1, Figure 1(b), p. 1]

因此：

$$\frac{890}{3514} \approx 0.253. \quad (9.17)$$

即约为前者的四分之一。

这里还有一个反直觉点：V4.1 并不是把时间维压缩做得越来越激进。它的 decoder 甚至使用 $m = 1$ ，保留每个原始位置的 global entry。真正节省下来的，是大量跨层重复缓存，再叠加更低的存储精度。[1, §4.2.1, p. 22; §2.4.4, p. 14]

所以不能只看“两个 token 合一个”就觉得压缩不够；省掉一层又一层重复保存的历史，影响非常大。

9.3.4 那 1/8 又是哪里来的？

1/8 指的是 persistent KV cache，即为了后续 prefix reuse 而长期保存在 SSD 或 host memory 中的缓存。

报告说，V4 的生产部署中，persistent cache 里 global KV 和 SWA KV 各占了接近一半。SWA 虽然每份只存一个窗口，但会在 prompt 末尾、output 末尾保存快照；大量短轮次会话累计起来，这部分并不小。[1, §3.2.1, p. 19]

设旧系统：

$$M_{\text{old}} = G + S, \quad S \approx G. \quad (9.18)$$

V4.1 做两件事：global KV 缩成约 $G/4$ ；SWA 不再进入这个长期 persistent cache，而靠短期缓存和 bounded replay 恢复。

于是：

$$M_{\text{new}} \approx \frac{G}{4}, \quad (9.19)$$

$$\frac{M_{\text{new}}}{M_{\text{old}}} \approx \frac{G/4}{G+G} = \frac{1}{8}. \quad (9.20)$$

这就是报告明确给出的两个乘法因素。[1, §3.2.1, p. 19]

9.3.5 SWA 不是完全不存，而是不再长期存

V4.1 仍然需要当前生成过程中的 SWA KV。

部署时，encoder SWA 状态放到一个短 TTL 的分布式主存池中；报告配置使用每台机器约 10% 的 host DRAM，保留时间为分钟级。Global KV 则继续长期保存，报告中的保留期至少为 72 小时。短期 SWA 缓存失效时，再通过 bounded replay 恢复。[1, §3.2.1, p. 19]

1/4 更接近模型架构和精度配置决定的 global KV 比例；
1/8 还依赖部署工作负载、SWA 快照占比与缓存策略。

不能把“1/8”解释为同一个 global KV entry 又被额外压缩了一半，也不能认为任意工作负载、任意缓存实现都会恰好得到八倍收益。

9.4 最后的判断：真正值得关注的创新是什么？

在我看来，这份报告最有价值的是不再要求所有历史信息、所有网络深度、所有存储介质遵循同一种处理方式。

长期历史信息，用少量跨层共享的 global KV 保存；局部细节，保留每层自己的 SWA；缺失的局部状态，接受小规模近似重建；历史 prompt，不必全部经过完整 decoder；当前预测位置，则仍然享有 40 层的深度。

但这些收益也有边界。CSA2 的候选筛选可能漏掉关键历史，bounded replay 恢复的状态并不精确，FP4 存储也存在量化误差。报告承认，虽然内部评估没有观察到系统性退化，但极端长上下文检索与缓存恢复边界仍需进一步验证。[1, §6, p. 37]

把整套架构压缩成一句话就是：用约半个网络把“过去”写成紧凑、可复用的记忆，用完整网络计算“现在”，再通过稀疏读取、低精度存储和推测解码，把这套分工转化为服务效率。

附录 A 符号、公式与原报告定位

A.1 主要符号与统计口径

本文并没有强行把所有前作的符号改成同一种约定。普通 Transformer 回顾中采用行向量投影，而 MLA、Engram 等前作示意沿用列向量写法；两者是转置约定的差别，不是架构差别。

符号／术语	本文的含义
T 或 N	上下文中的原始位置数量；基础回顾常用 T ，报告架构记账常用 N 。
L	Transformer 层数；本模型语言主干 $L = 40$ 。
B	容量公式中为并发请求数；mHC 中 B_l 是残差混合矩阵，DSpark 中 B 是转移修正矩阵，按所在章节区分。
h_Q, h_{KV}	Query heads 与 KV heads 数量，不应在 GQA/MQA 中混为一谈。
d, d_h	Hidden dimension 与 head dimension。Main KV latent 的实际存储维度未必等于 $h_Q d_h$ 。
s	基础容量公式中每个数值的字节数；replay 公式中为重放起始位置，语境不同。
W 或 n_{win}	SWA 窗口；本模型为 128。视觉网格公式中的 W 则表示宽度。
m	时间压缩率；encoder 的 CSA2 为 2，decoder 为 1。
Global KV	Main KV 与 Indexer K 的合计，不包含 SWA KV。
Persistent KV	为 prefix reuse 而长期保存的缓存；它是部署生命周期口径，不等同于全部运行时缓存。
8B / 16B	报告中的 prefill / decode 激活参数规模，不是两个独立模型的总参数量。
GiB 与 GB	$1 \text{ GiB} = 2^{30} \text{ bytes}$ ； $1 \text{ GB} = 10^9 \text{ bytes}$ 。本文的假设模型容量表用 GiB，890 MB 示例用十进制。

A.2 关键公式快速定位

问题	对应公式
标准 attention 的 Q/K/V 与输出	式 (1.1)–(1.4)
普通 KV cache 的完整容量	式 (1.8)、(1.9)
压缩、共享、局部缓存统一记账	式 (1.21)
CED 的上层 global KV 从哪里来	式 (3.2)
CSA2 为什么只维护 $2.5N$ 个 global entries	式 (4.8)
后续索引受限，但总体不是严格 $O(1)$	式 (4.10)
Single-Pass mHC 的错位混合	式 (5.5)、(5.6)
DSpark 的前缀存活与接受长度	式 (5.13)、(5.14)
Bounded Replay 的截断范围	式 (7.3)
Prefill 接近一半的层—token 计数	式 (9.4)
890 bytes/token 的完整复原	式 (9.8)、(9.11)、(9.14)
1/4 与 1/8 的不同统计口径	式 (9.17)、(9.20)

A.3 原报告的阅读路径

表 A.2 以下均为上传原报告的页码

主题	页码	优先阅读的位置
模型整体与全局 KV	1、4–7	Abstract、Figures 1–3、Introduction
CED	8–9	§2.1 概述、§2.2 与原文式 (1)
CSA2 三种模式	9–11	§2.3.1 与 Figure 4
Hierarchical Indexer	11–12	§2.3.2 与 Figure 5
Single-Pass mHC	12–13	§2.4.1, 原文式 (2)–(6)
Engram / DSpark	13–14	§2.4.2–§2.4.3
FP4 Main KV	14	§2.4.4
优化器与训练共享机制	15–18	§2.5、Algorithm 1、§3.1.2
部署与 SWA Replay	18–20	§3.2.1–§3.2.2
精确模型配置	21–22	§4.2.1
FLOPs 比较的边界	5、24	Figure 2 的精度加权说明、Table 1 的激活参数
稳健性与未覆盖边界	37	§6

参考文献

- [1] DeepSeek-AI. *DeepSeek-V4.1-Flash: Pushing the Limits of KV Cache Compression*. 用户上传的技术报告, 文件名 `DeepSeek_V41_Tech_Report.pdf`, 51 页。本文的“原报告页码”均以此版本为准。
- [2] Vaswani, A., et al. *Attention Is All You Need*. 2017. [arXiv:1706.03762](#). 基础 attention、多头投影与分层因果结构。
- [3] Shazeer, N. *Fast Transformer Decoding: One Write-Head is All You Need*. 2019. [arXiv:1911.02150](#). Multi-Query Attention 与增量解码带宽。
- [4] Ainslie, J., et al. *GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints*. 2023. [arXiv:2305.13245](#). Query heads 分组共享 KV。
- [5] Hugging Face. *Caching*. Transformers 官方文档, 访问于 2026 年 9 月 11 日。缓存机制说明。用于补充普通自回归缓存的逐层、逐位置执行语义。
- [6] DeepSeek-AI. *DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model*. 2024. [arXiv:2405.04434](#). MLA、低秩 KV 与解耦 RoPE。
- [7] DeepSeek-AI. *DeepSeek-V3.2: Pushing the Frontier of Open Large Language Models*. 2025. [arXiv:2512.02556](#). DSA、轻量索引器与稀疏选择。
- [8] DeepSeek-AI. *DeepSeek-V4: Towards Highly Efficient Million-Token Context Intelligence*. 2026. [arXiv:2606.19348](#). CSA/HCA、Shared Key-Value MQA、compressor 与分组输出投影。
- [9] Sun, Y., et al. *You Only Cache Once: Decoder-Decoder Architectures for Language Models*. 2024. [arXiv:2405.05254](#). 下层构造共享缓存与上层读取的架构。
- [10] Brandon, W., Mishra, M., Nrusimha, A., Panda, R., and Ragan-Kelley, J. *Reducing Transformer Key-Value Cache Size with Cross-Layer Attention*. 2024. [arXiv:2405.12981](#). 跨层 attention 的 KV 共享。
- [11] Xie, Z., et al. *mHC: Manifold-Constrained Hyper-Connections*. [arXiv:2512.24880](#). 多残差流、双随机混合及稳定性约束的前作。
- [12] Cheng, X., et al. *Conditional Memory via Scalable Lookup: A New Axis of Sparsity for Large Language Models*. 2026. [arXiv:2601.07372](#). Engram 的 tokenizer compression、多头 hashing 与上下文 gating。
- [13] Cheng, X., et al. *DSpark: Confidence-Scheduled Speculative Decoding with Semi-Autoregressive Generation*. 2026. [arXiv:2607.05147](#). 并行基础预测、Markov head 与自适应验证。
- [14] Dai, D., et al. *DeepSeekMoE: Towards Ultimate Expert Specialization in Mixture-of-Experts Language Models*. 2024. [arXiv:2401.06066](#). 细粒度专家与 shared expert。
- [15] Alvarez, E., et al. NVIDIA. *Introducing NVFP4 for Efficient and Accurate Low-Precision Inference*. 2025. [NVIDIA 官方技术说明](#). NVFP4 与 MXFP4 的块尺度及格式比较。
- [16] Dao, T., et al. *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness*. 2022. [arXiv:2205.14135](#). Attention 中间矩阵与 I/O 优化, 不等同于缩小逻辑 KV 状态。
- [17] Kwon, W., et al. *Efficient Memory Management for Large Language Model Serving with PagedAttention*. 2023. [arXiv:2309.06180](#). 分页式 KV 管理、碎片控制与共享。
- [18] Jiang, A. Q., et al. *Mistral 7B*. 2023. [arXiv:2310.06825](#). GQA、滑动窗口注意力与滚动缓存背景。